

Tutorial NodeJS

*Conectando uma aplicação Nodejs
com Banco de dados MySQL*

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

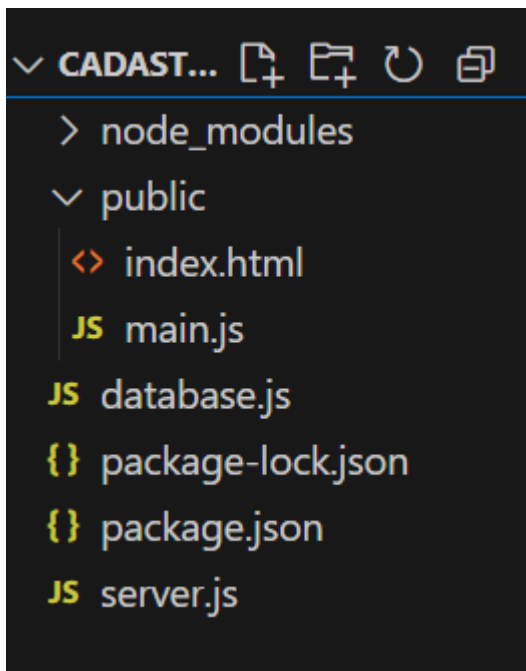
Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Tutorial: Projeto com JavaScript puro (frontend) e Node.js no backend , utilizando banco de dados MySQL para armazenar os cadastros.

Objetivo:

```
cadastro-usuario/  
├── public/  
│   ├── index.html  
│   └── main.js  
├── server.js  
├── package.json  
└── database.js
```

Vamos criar uma pasta para colocarmos o projeto:



Abram com o vscode.

1. Passo: Configuração Inicial

Instale as dependências:

```
npm init -y  
npm install express mysql body-parser cors
```

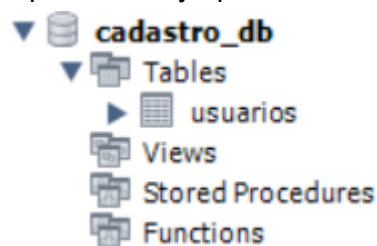
“você também pode utilizar o **mysql2** no lugar do mysql

2. Banco de Dados

Crie uma tabela no MySQL:

```
CREATE DATABASE cadastro_db;  
  
USE cadastro_db;  
  
CREATE TABLE usuarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100)  
);
```

Aqui usei o Mysql Workbench para criar o banco e a tabela usuarios:



3. Arquivo database.js – Conexão com o Banco

```
const mysql = require('mysql');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'cadastro_db'
});

connection.connect((err) => {
  if (err) throw err;
  console.log('Conectado ao banco de dados MySQL');
});

module.exports = connection;
```

Explicando as linhas de códigos:

const mysql , está chamando a lib de conexão de banco de dados mysql que instalamos anteriormente.

const connection vai criar a conexão da aplicação com o banco de dados mysql no caso, onde iremos preencher com os dados da aplicação, host = localhost, user = nome do usuário do seu banco de dados no meu caso (root), password = senha do seu banco de dados (no meu caso senha em branco) e database = nome do seu banco de dados (no meu caso **cadastro_db**).

Na sequência temos um bloco de detecção de erros ao tentar conectar no banco de dados e depois exportamos para chamar a conexão em outro arquivo.

4. Arquivo server.js – Backend com Express

```

JS server.js > ...
1  const express = require('express');
2  const bodyParser = require('body-parser');
3  const cors = require('cors');
4  const db = require('./database');
5
6  const app = express();
7  app.use(cors());
8  app.use(bodyParser.json());
9  app.use(express.static('public'));
10
11  // Rota para cadastrar usuário
12  app.post('/cadastrar', (req, res) => {
13    const { nome, email } = req.body;
14    const sql = 'INSERT INTO usuarios (nome, email) VALUES (?, ?)';
15    db.query(sql, [nome, email], (err, result) => {
16      if (err) return res.status(500).json({ error: err.message });
17      res.json({ message: 'Usuário cadastrado!', id: result.insertId });
18    });
19  });
20
21  // Iniciar servidor
22  const PORT = 3000;
23  app.listen(PORT, () => {
24    console.log(`Servidor rodando na porta ${PORT}`);
25  });

```

Explicação detalhada:

express: Framework para construir servidores HTTP de forma simples e rápida.

body-parser: Middleware para interpretar o corpo das requisições (em JSON, no caso).

cors: Middleware para permitir requisições de diferentes origens (Cross-Origin Resource Sharing), essencial para aplicações frontend consumirem essa API.

db: Arquivo local (./database) responsável pela conexão com o banco de dados.

app.use(cors()): Libera o acesso da API para qualquer origem (útil em APIs públicas ou frontends rodando em domínios diferentes).

app.use(bodyParser.json()): Faz o parse (interpretação) do corpo da requisição no formato JSON.

app.use(express.static('public')): Permite servir arquivos estáticos (imagens, CSS, HTML, etc.) que estiverem na pasta public.

app.post('/cadastrar'): Cria uma rota HTTP POST no caminho /cadastrar.

const { nome, email } = req.body; Desestrutura os dados que vieram no corpo da requisição JSON. Exemplo de JSON esperado:

sql: Comando SQL para inserir os dados na tabela usuarios.

db.query(): Executa o comando SQL usando os valores [nome, email]. Usa ? como placeholders para evitar SQL Injection.

Callback: Se houver erro, responde com status 500 e a mensagem do erro. Se funcionar, responde com um JSON de sucesso contendo a mensagem e o ID do usuário cadastrado.

Define a porta (**3000**) onde o servidor irá escutar.

O callback do **listen** apenas imprime no console que o servidor está rodando.

5. Arquivo public/index.html – Frontend HTML

```
public > <> index.html > html
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4    <meta charset="UTF-8" />
5    <title>Cadastro de Usuário</title>
6  </head>
7  <body>
8    <h2>Cadastrar Usuário</h2>
9    <form id="formCadastro">
10     <label>Nome:</label><br />
11     <input type="text" id="nome" required /><br /><br />
12
13     <label>Email:</label><br />
14     <input type="email" id="email" required /><br /><br />
15
16     <button type="submit">Cadastrar</button>
17   </form>
18
19   <p id="mensagem"></p>
20
21   <script src="main.js"></script>
22 </body>
23 </html>
```

6. Arquivo public/main.js – Lógica do Frontend (JavaScript Puro)

```
public > JS main.js > ...
1 document.getElementById('formCadastro').addEventListener('submit', function(e) {
2     e.preventDefault();
3
4     const nome = document.getElementById('nome').value;
5     const email = document.getElementById('email').value;
6
7     fetch('http://localhost:3000/cadastrar', {
8         method: 'POST',
9         headers: {
10             'Content-Type': 'application/json'
11         },
12         body: JSON.stringify({ nome, email })
13     })
14     .then(response => response.json())
15     .then(data => {
16         document.getElementById('mensagem').textContent = data.message;
17     })
18     .catch(error => {
19         document.getElementById('mensagem').textContent = 'Erro ao cadastrar usuário.';
20         console.error('Erro:', error);
21     });
22 });
```

Este trecho de código é JavaScript executado no navegador, responsável por capturar os dados de um formulário HTML, enviar para o backend (a API que você mostrou antes) e exibir uma mensagem com o resultado.

Explicação do código:

document.getElementById('formCadastro'); Seleciona o formulário que tem o id="formCadastro".

addEventListener('submit', function(e) {...}); Adiciona um evento que dispara quando o formulário é enviado.

e.preventDefault(); Impede que o formulário recarregue a página (comportamento padrão do HTML).

Captura os valores dos campos de entrada que possuem os IDs **nome** e **email**.

fetch(); Função nativa do navegador que faz requisições **HTTP**.

URL: Está apontando para o backend rodando localmente (**http://localhost:3000/cadastrar**).

method: 'POST'; Método **HTTP POST** para envio de dados.

headers: Define que o corpo da requisição está em **JSON**.

body: Converte o objeto { nome, email } em JSON usando **JSON.stringify()**.

response.json(): Converte a resposta da API (que vem em JSON) para um objeto JavaScript.

O **then(data => {...})** executa quando a resposta chega com sucesso.

Atualiza o conteúdo do elemento com ID mensagem para exibir a mensagem retornada pela API (ex.: “**Usuário cadastrado!**”).

Se ocorrer qualquer erro (**API** fora do ar, problemas de rede, etc.), exibe uma mensagem de erro na página e também imprime o erro no console do navegador.

Você executa esse servidor com:

Certifique-se de estar com a conexão com banco de dados ativa e rode o seguinte comando no terminal da sua aplicação:

```
$ node server.js
Servidor rodando na porta 3000
Servidor rodando na porta 3000
Conectado ao banco de dados MySQL
```

Se tudo correr bem, vai aparecer esta mensagem, caso contrário procure verificar o tipo de mensagem de erro que está ocorrendo normalmente vai ser a conexão do banco de dados ou algum erro nas variáveis de conexão com o banco de dados.

Com isso podemos acessar **http:localhost:3000** e vejamos se vai aparecer assim :



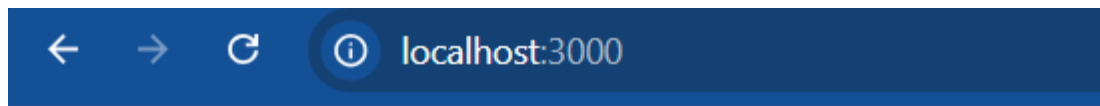
Cadastrar Usuário

Nome:

Email:

Cadastrar

Vamos testar o cadastro de um usuário agora e verificar se está salvando no banco de dados.



Cadastrar Usuário

Nome:



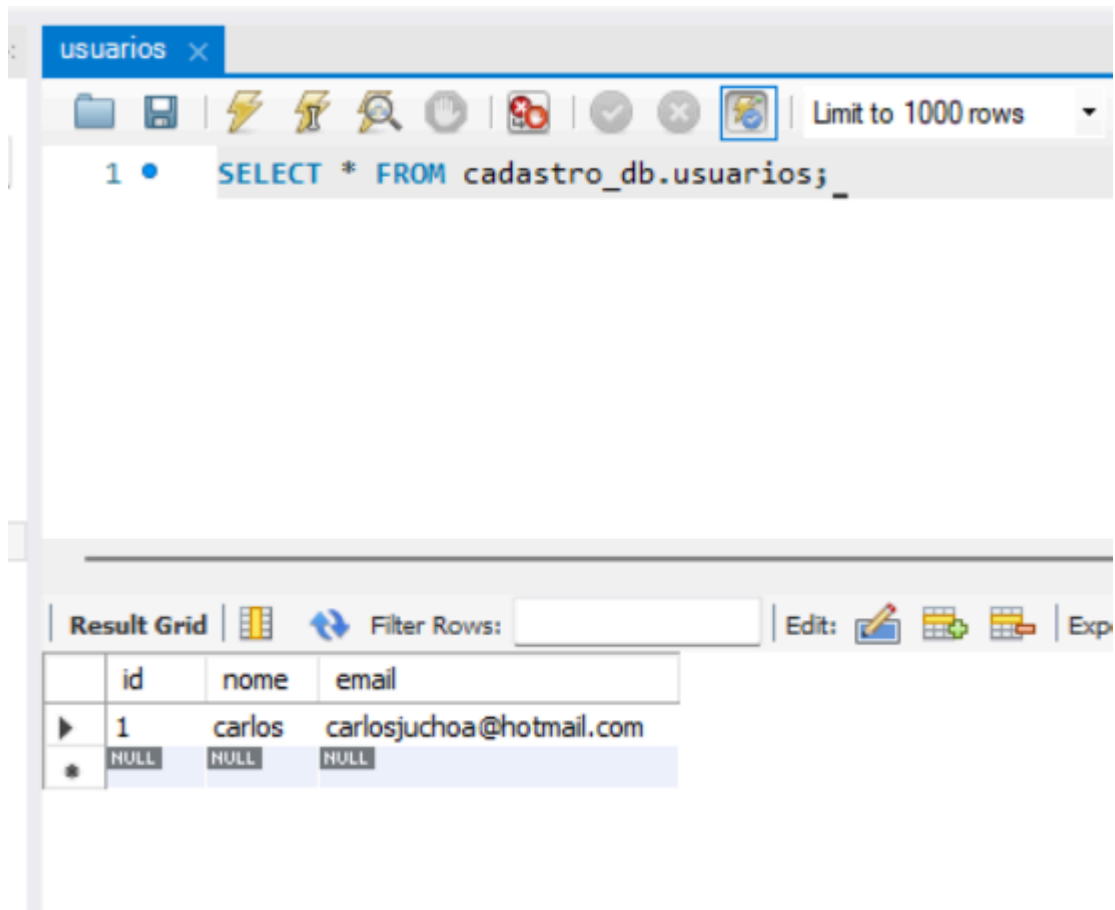
Email:



Usuário cadastrado!

Colocamos informações nos campos e clicamos no botão cadastrar, apareceu a mensagem de **Usuário cadastrado!** é nossa deixa para verificarmos no banco de dados se realmente cadastrou.

Acessando o banco de dados:



Como podem ver deu certo nossa conexão com banco de dados e conseguimos gravar as informações corretamente neste mini projeto.